ASSIGNMENT 5 – CO5

RELIABLE SOFTWARE WITH RUNTIME POLYMORPHISM AND EXCEPTIONS

NAME: DENISHA SHREE R

REG NO: 192524016

Course Outcome – CO5: Build robust C++ applications by applying virtual functions, runtime polymorphism and exception handling mechanisms to develop reliable and error resilient software solutions (virtual and pure virtual functions, runtime polymorphism, throwing, catching and rethrowing exceptions, and specifying exceptions)

Nature: Individual

Duration: One Week

Total Marks: 50

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5

level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment5 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it

and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.
Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assessment Components

PART A – Design: 25 Marks

PART B – Implementation & Experimental Investigation: 25 Marks

PURPOSE OF THE ASSIGNMENT

Large programs must handle many kinds of objects through a common interface, and must keep working – or fail safely – when something goes wrong.

This assignment investigates a fundamental question:

How should a C++ programmer design a program so that new kinds of objects can be added without changing working code, and errors are reported and handled without losing data or resources?

You are expected to demonstrate the following progression:

Abstract → Specialise → Bind at Run Time → Detect Errors → Throw → Catch → Recover → Measure → Decide

Simply obtaining a program that runs is not sufficient. Your design reasoning, experimental evidence and interpretation are all part of the assessment.

Case Study — Hospital Billing and Insurance Claims

A multi-speciality hospital raises different kinds of bills: out-patient (consultation and tests), in-patient (room charges per day, procedures and medicines), day-care surgery (package price) and emergency (base charge plus time-based charges). Bills may be paid by the patient, by an insurance company within the policy cover, or by a government health scheme with fixed package rates. The billing system must compute every bill correctly, reject invalid claims with clear reasons, and never lose a bill because of an error.

PART A – DESIGN 25 Marks

All pseudocode must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode.

TASK A1 – DESIGN THE POLYMORPHIC BILL HIERARCHY 7

Marks Bloom's Level: K5 – Evaluate

A1.1 – Identify the Common Interface | 2 Marks
Identify the operations that every bill must provide (for example: total(), type(), print()) and the data common to all bills.

A1.2 – Design the Hierarchy | 3 Marks

Design an abstract base class Bill with pure virtual functions, and derived classes for each kind of bill. Draw the UML class diagram and mark which functions are virtual, pure virtual and overridden.

A1.3 – Process All Bills Together | 2 Marks

Write pseudocode that computes the day's total for an array of base-class pointers to bills of every kind.

Alternative Concept and Complexity Analysis

If the bills were represented by one Bill structure with a type field and a switch in total(), instead of an abstract base class with virtual functions, what would the solution be? Analyse the time to total n bills (O(n) in both, with a switch or a virtual call per bill) and the space per bill, and the code changed when a new bill type is added.

Evidence Expected

A common interface, a UML hierarchy with virtual functions marked, and pseudocode for processing all bills.

TASK A2 – UNDERSTAND RUNTIME BINDING5 Marks

Bloom's Level: K5 – Evaluate

A2.1 – Early and Late Binding | 2 Marks

Explain the difference between compile-time (early) and run-time (late) binding using a call through a Bill pointer.

A2.2 – The Virtual Table | 2 Marks

Draw the virtual tables (vtables) of Bill, OutPatientBill and InPatientBill, and show how a call b->total() reaches the right function.

A2.3 – Virtual Destructors | 1 Mark

Explain why Bill must have a virtual destructor.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>
struct Bill {
 virtual ~Bill() { std::cout << "~Bill "; }
 virtual double total() const = 0;
 void show() const { std::cout << total() << ' '; }
};

struct OPBill : Bill {
 ~OPBill() override { std::cout << "~OPBill "; }
 double total() const override { return 500; }
};

int main()
{
 Bill* b = new OPBill;
 b->show();
 delete b;
 std::cout << '\n';
}
```

My predicted output (write before running):

Explain how show(), a non-virtual function, reached OPBill::total(). Then remove virtual from ~Bill, predict the new output and verify it under -fsanitize=address.

Alternative Concept and Complexity Analysis

If total() were not virtual and the program called the correct derived function by testing the

type of each bill (with dynamic_cast), what would the solution be? Analyse the time per bill (one virtual call versus up to k casts for k bill types) and the space used by the vtable pointer in each object.

Evidence Expected

Correct explanations of binding, vtable diagrams and the role of the virtual destructor.

TASK A3 – DESIGN WITH ABSTRACT CLASSES 5 Marks

Bloom's Level: K5 – Evaluate

A3.1 – Payer Interface | 3 Marks

Design an abstract class Payer with a pure virtual function amountCovered(const Bill&), and derived classes SelfPay, Insurance (cover limit and co-payment percentage) and GovernmentScheme (fixed package rates).

A3.2 – Extension | 2 Marks

Show how a new payer (for example a corporate tie-up) can be added without changing any existing class.

Alternative Concept and Complexity Analysis

If the payers were represented by function pointers or std::function objects (a policy function chosen at run time) instead of an abstract Payer class, what would the solution be? Analyse the time per call (O(1) in both) and the space per payer, and compare how each design stores data such as the cover limit.

Evidence Expected

An abstract payer design with derived classes and a demonstrated extension path.

TASK A4 – DESIGN THE EXCEPTION STRATEGY 4 Marks

Bloom's Level: K5 – Evaluate

A4.1 – Identify the Errors | 2 Marks

List every error that can occur (negative charges, claim above cover, unknown package, missing patient record, memory allocation failure) and design a hierarchy of exception classes derived from std::exception (for example BillingError, ClaimError).

A4.2 – Specify Exceptions | 2 Marks

Decide which functions may throw and which must never throw, and mark the latter noexcept. Explain why destructors should not throw.

Alternative Concept and Complexity Analysis

If errors were reported by return codes (an enum Status or a bool with an error message) instead of exceptions, what would the solution be? Analyse the time on the normal path and on the error path, and the extra space and code needed to pass every error back to the caller.

Evidence Expected

An error list, an exception class hierarchy and justified exception specifications.

TASK A5 – DESIGN RECOVERY AND RETHROWING 4 Marks

Bloom's Level: K5 – Evaluate

A5.1 – Where to Catch | 2 Marks

Decide where each exception is caught, where it is logged and rethrown with throw;, and where the program finally reports it to the user.

A5.2 – No Leaks on Errors | 2 Marks

Explain stack unwinding, and how objects with destructors (RAII) ensure that memory and files are released when an exception passes through a function.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>
#include <string>

struct Guard {
 std::string n;
 explicit Guard(const std::string& s) : n(s) {}
 ~Guard() { std::cout << '~' << n << ' '; }
};

void f()
{
 Guard g("f");
 throw 42;
}
```

```
int main()
{
 try {
 Guard m("main");
 f();
 std::cout << "not printed ";
 } catch (int x) {
 std::cout << "caught " << x;
 } catch (...) {
 std::cout << "other";
 }
 std::cout << '\n';
}
```

My predicted output (write before running):

Explain the order in which the destructors ran and why "not printed" never

appears. Alternative Concept and Complexity Analysis

If memory were released by delete in every catch block, instead of by destructors (RAII objects such as std::unique_ptr or std::vector), what would the solution be? Analyse the time (the same) and the code size, and count the places where a leak can occur when a new error path is added.

Evidence Expected

A catch-and-rethrow design and a correct explanation of stack unwinding and resource safety.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION 25 Marks

Part B assesses whether your design reasoning survives contact with actual computation. You are expected to:

Implement → Instrument → Stress → Measure → Compare → Interpret

Compile with g++ -std=c++17 -Wall -Wextra and run the tests with -fsanitize=address. Execution screenshots alone are insufficient.

TASK B1 – IMPLEMENT AND VALIDATE THE POLYMORPHIC BILLING 7 Marks

Bloom's Level: K5 – Evaluate

B1.1 – Validate the Implementation | 2 Marks

Implement the hierarchy from A1 and A3 and test one bill of each kind with each payer. Verify the totals by hand.

B1.2 – Compare with Your Design | 1 Mark

Compare your code with your Part A design and explain any change.

B1.3 – Prove Runtime Binding | 2 Marks

Remove the virtual keyword from total() and record how the output changes when bills are processed through base-class pointers.

B1.4 – Prove the Virtual Destructor | 2 Marks

Remove the virtual destructor, run under the address sanitizer and record the

result. Complete the Code

Complete TODO 1 to TODO 3 below, and execute the program.

```
#include <iostream>

#include <string>


class Bill {

protected:

 std::string patient;


public:

 explicit Bill(const std::string& p) : patient(p) {}

 virtual ~Bill() = default;
 // TODO 1: Declare a pure virtual function total() const returning double, and a

// virtual function type() const returning "Bill"
```

```
 void print() const
 {
 std::cout << type() << ' ' << patient << ": " << total() << '\n';  }
};

class OutPatient : public Bill {
 double consultation, tests;

public:
 OutPatient(const std::string& p, double c, double t)
 : Bill(p), consultation(c), tests(t) {}
 // TODO 2: Override total() as consultation + tests, and type() as "OP"
};

class InPatient : public Bill {
 int days;
 double perDay, surgery;

public:
 InPatient(const std::string& p, int d, double rate, double s)
 : Bill(p), days(d), perDay(rate), surgery(s) {}
 // TODO 3: Override total() as days × perDay + surgery, and type() as "IP"

};

int main()
{
 Bill* bills[] = {new OutPatient("Ravi", 500, 1200),
 new InPatient("Meena", 3, 2500, 40000)};
```

```
double sum = 0;
for (Bill* b : bills) {
b->print(); // runtime polymorphism
sum += b->total();
}
std::cout << "Total " << sum << '\n';
for (Bill* b : bills) delete b; // virtual destructor runs the right destructor }
```

Expected output:

OP Ravi: 1700

IP Meena: 47500

Total 49200

Remove virtual from total() and explain the compiler error you

receive. Alternative Concept and Complexity Analysis

If the bills were processed with templates (compile-time polymorphism) for each bill type in its own container, instead of virtual functions through a base-class pointer, what would the solution be? Analyse the time per bill (no virtual call) and the space (one container per type), and what is lost when bills of all types must be kept in one list.

Evidence Expected

Working code, a verified test table, and evidence of what virtual functions and virtual destructors change.

TASK B2 – ENGINEER ERROR CONDITIONS 5 Marks

Bloom's Level: K5 – Evaluate

• Feed the system negative charges, a claim above the cover, an unknown package code and text where a number is expected (std::stod throws std::invalid_argument)

• Force a memory allocation failure (for example by requesting an impossibly large array) and catch std::bad_alloc

• Show that a catch(...) at the top level keeps the program running and that all objects are destroyed (print messages in destructors)

• Explain what each test reveals about your exception design

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

#include <stdexcept>


void validate(double amount)

{

 if (amount < 0) throw std::invalid_argument("negative");

 if (amount > 1e6) throw std::out_of_range("too large");

 std::cout << "ok ";

}


void process(double amount)

{

 try {

 validate(amount);

 } catch (const std::out_of_range& e) {

 std::cout << "[log " << e.what() << "] ";
 throw;

 }

}


int main()

{

 for (double a : {100.0, -5.0, 2e6}) {

 try {

 process(a);

 std::cout << "done ";

 } catch (const std::exception& e) {
```

```
 std::cout << "caught " << e.what() << ' ';
 }
 }
 std::cout << '\n';
}
```

My predicted output (write before running):

Explain which handler caught each exception and why only one exception was logged before being rethrown.

Complete the Code

Complete TODO 1 and TODO 2 below, and execute the program.

```
#include <iostream>
#include <stdexcept>
class ClaimError : public std::runtime_error {
public:
 using std::runtime_error::runtime_error;
};

double approve(double billed, double cover)
{
 // TODO 1: Throw std::invalid_argument("negative amount") if billed or cover is  //
negative; throw ClaimError("exceeds cover") if billed > cover; otherwise return  //
billed

}
```

```
double settle(double billed, double cover)
 {
 try {
 return approve(billed, cover);
 } catch (const ClaimError& e) {
 // TODO 2: Print "audit: " followed by the message, then rethrow the SAME  //
exception

 }
 }

int main()
 {
 double cases[][2] = {{40000, 50000}, {60000, 50000}, {-1, 50000}};
for (auto& c : cases) {
 try {
 double paid = settle(c[0], c[1]);
 std::cout << "approved " << paid << '\n';
 } catch (const ClaimError& e) {
 std::cout << "claim rejected: " << e.what() << '\n';
 } catch (const std::exception& e) {
 std::cout << "error: " << e.what() << '\n';
 }
 }
 }
```

Expected output:

```
approved 40000
audit: exceeds cover
```

claim rejected: exceeds cover

error: negative amount

Replace throw; with throw e; and explain the difference (hint: the type of the rethrown object).

Alternative Concept and Complexity Analysis

If invalid claims were rejected by std::optional<double> (no value means rejected) instead of by throwing exceptions, what would the solution be? Analyse the time for 1,000,000 claims with 0%, 1% and 50% invalid claims in both designs, and the extra space per result.

Evidence Expected

Deliberately constructed error conditions, observed behaviour and a reasoned explanation.

TASK B3 – VIRTUAL FUNCTIONS vs switch vs EXCEPTIONS: THE COST 8 Marks

Bloom's Level: K5 – Evaluate

B3.1 – Correctness on Small Data | 2 Marks
Compute the totals of the same 10 bills using (a) virtual functions and (b) a single Bill struct with a type field and a switch statement. Verify that both agree.

B3.2 – Increase the Data Size | 2 Marks

Measure the time of both versions for 1,000, 1,000,000 and 10,000,000 bills, compiled with -O2.

B3.3 – Cost of Exceptions | 2 Marks

Measure the time to validate 1,000,000 bills when errors are reported by return codes and when they are reported by exceptions, with 0%, 1% and 50% invalid bills.

B3.4 – Engineering Trade-Off | 2 Marks

Explain the results, and state when each approach should be used.

Alternative Concept and Complexity Analysis

If the bill types were held in std::variant<OutPatientBill, InPatientBill, …> and processed with std::visit, instead of virtual functions or a switch, what would the solution be? Measure the time for 10,000,000 bills against your two versions, and analyse the space per element (the size of the largest alternative plus a tag).

Evidence Expected

Verified correctness, measured comparisons and a reasoned trade-off.

TASK B4 – FINAL ENGINEERING DECISION 5 Marks

Bloom's Level: K5 – Evaluate

The hospital group now runs 15 hospitals, adds two new bill types and three new payers every year, and processes 200,000 bills a day.

B4.1 – Implement a Practical Extension | 2 Marks

Add one new bill type and one new payer to your program without changing any existing class, and show that all earlier tests still pass.

B4.2 – Check Resource Safety | 1 Mark

Show a clean address-sanitizer report for a run that includes thrown and caught

exceptions. B4.3 – Production Decision | 2 Marks

Decide whether the polymorphic design with exceptions is suitable for the hospital group, using your measurements and the effort of extension. Justify your decision.

Alternative Concept and Complexity Analysis
If new bill types and payers were loaded as plug-ins (configuration files describing the rates) instead of being written as new C++ classes, what would the solution be? Analyse the time per bill (rule interpretation versus a virtual call) and the space for the rules, and compare the effort and risk of each kind of extension.

Evidence Expected
A working extension, a clean sanitizer report and an evidence-based production decision.
SUBMISSION REQUIREMENTS

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.

For Part A, submit the UML diagrams, vtable diagrams, exception hierarchy, pseudocode and reasoning. C++ code will not be accepted in place of pseudocode.

For Part B, submit C++ source code, test inputs, predictions, actual outputs, timings and sanitizer reports, with interpretation.

Students are expected to use the same engineering workflow throughout the assignment:

UNDERSTAND → ABSTRACT → DESIGN → IMPLEMENT → BREAK → MEASURE → INTERPRET

The key question is not simply "Does the program work?" It is also:

Can new kinds of bills and payers be added safely, and does the program stay correct and leak-free when things go wrong?

ANSWERS:

ASSIGNMENT 5 – CO5

RELIABLE SOFTWARE WITH RUNTIME POLYMORPHISM AND EXCEPTIONS

INTRODUCTION

This assignment focuses on developing a reliable hospital billing system using C++ concepts such as virtual functions, runtime polymorphism, abstract classes and exception handling.

The system supports different types of hospital bills and payment methods. It calculates bill amounts, validates insurance claims and handles errors safely.

The main objective is to create a flexible system where new bill types and payment methods can be added without changing the existing working code.

PART A – DESIGN:

TASK A1 – DESIGN THE POLYMORPHIC BILL HIERARCHY

A1.1 – Identify the Common Interface:

Every hospital bill should provide the following common operations:

1. **total():** Calculates the final bill amount.
2. **type():** Returns the type of the bill.
3. **print():** Displays the bill details.

The common data members are:

- Patient name or patient ID.
- Bill number.
- Bill date.
- Billing status.

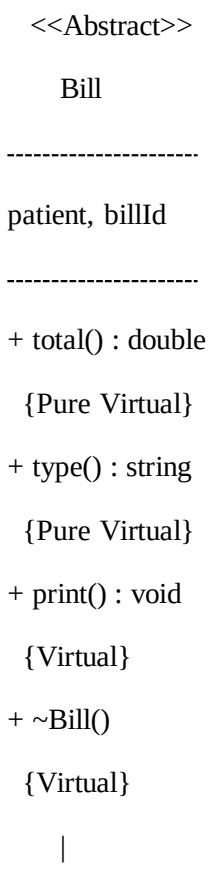
Different bill types require different calculations.

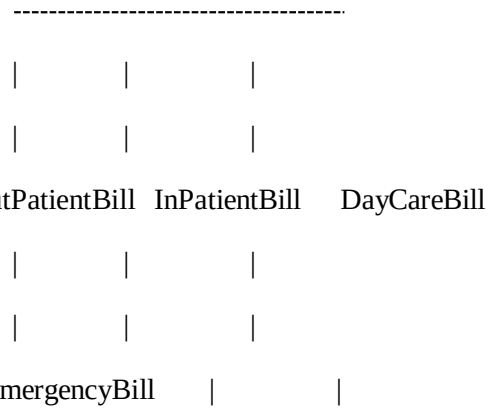
- **OutPatientBill:** Consultation fee + test charges.
- **InPatientBill:** Number of days × daily room charge + procedures + medicines.
- **DayCareBill:** Fixed package price.
- **EmergencyBill:** Base charge + hourly charge × number of hours.

A1.2 – Design the Hierarchy:

The base class Bill is designed as an abstract class. It contains pure virtual functions that must be implemented by all derived classes.

UML Class Diagram:





Derived Classes:

- OutPatientBill
- InPatientBill
- DayCareBill
- EmergencyBill

Each derived class overrides:

+ total()
+ type()

The Bill class cannot be instantiated directly because it contains pure virtual functions.

Each derived class provides its own implementation of total() and type().

The virtual destructor ensures that derived class objects are properly destroyed when deleted through a base-class pointer.

A1.3 – Process All Bills Together:

Algorithm: CalculateDailyTotal

Input: Array of base-class pointers containing different bill objects.

Output: Total amount of all bills.

ALGORITHM CalculateDailyTotal

BEGIN

dailyTotal ← 0

FOR i ← 0 TO n-1 DO

```
    IF bills[i] IS NOT NULL THEN
        dailyTotal ← dailyTotal + bills[i].total()
    END IF
  END FOR
  DISPLAY dailyTotal
END
```

This algorithm processes all types of bills through a common base-class pointer. Runtime polymorphism automatically calls the correct total() function.

Alternative Concept and Complexity Analysis:

If all bills were represented using a single structure with a type field, a switch statement would be used to calculate the total.

For example:

- If type is OP, calculate consultation + tests.
- If type is IP, calculate room charges + procedures + medicines.
- If type is DayCare, return the package price.
- If type is Emergency, calculate the emergency charges.

Both approaches take **O(n)** time to calculate the total of n bills.

In the virtual-function approach, each object generally contains a hidden virtual table pointer. In the structure approach, each bill stores a type field and the required data.

When a new bill type is added, the switch statement and central structure must be modified. In the polymorphic approach, a new derived class can be added without changing the existing processing loop.

TASK A2 – UNDERSTAND RUNTIME BINDING

A2.1 – Early Binding and Late Binding:

Early Binding:

Early binding means that the function call is decided during compile time. It is generally used with non-virtual functions.

Late Binding:

Late binding means that the function call is decided during program execution. It

is achieved using virtual functions.

Example:

```
Bill* b = new OutPatientBill(...);

b->total();
```

Here, b is a base-class pointer, but it points to an OutPatientBill object.

Since total() is virtual, the program calls OutPatientBill::total() at runtime.

Therefore, late binding supports runtime polymorphism.

A2.2 – Virtual Table:

A virtual table, commonly called a vtable, is an implementation technique used by C++ compilers to support virtual function calls.

Conceptual Vtable Diagram:

```
Bill Vtable

---------------------------

Destructor → Bill::~Bill

total()    → Pure virtual slot

---------------------------

OutPatientBill Vtable

---------------------------

Destructor → OutPatientBill::~OutPatientBill

total()    → OutPatientBill::total

---------------------------

InPatientBill Vtable

---------------------------

Destructor → InPatientBill::~InPatientBill

total()    → InPatientBill::total

---------------------------
```

How the call works:

1. A base-class pointer points to a derived-class object.

2. The object contains a hidden pointer to its virtual table.
3. The program identifies the correct virtual function.
4. The derived-class function is executed.

For example, b->total() calls OutPatientBill::total() when b points to an OutPatientBill object.

The exact internal layout of a vtable depends on the compiler.

A2.3 – Virtual Destructor:

A virtual destructor is important when deleting derived-class objects using base-class pointers.

If the base-class destructor is not virtual, deleting a derived object through a base pointer causes undefined behaviour.

A virtual destructor ensures that the derived-class destructor runs first, followed by the base-class destructor.

Therefore, the base class Bill must have a virtual destructor.

Predicted Output:

500 ~OPBill ~Bill

Explanation:

1. b->show() calls the non-virtual show() function.
2. Inside show(), the virtual function total() is called.
3. Runtime binding selects OPBill::total().
4. Therefore, the value 500 is displayed.
5. When delete b executes, the derived destructor runs first.
6. The base destructor runs next.

The final output is:

500 ~OPBill ~Bill

What happens if virtual is removed from the destructor?

Removing virtual from the base destructor makes deletion through a base-class pointer undefined behaviour.

There is no guaranteed output in this situation. The derived destructor may not run correctly.

The program should be tested using AddressSanitizer. The exact sanitizer result depends on the compiler and runtime environment.

Alternative Concept and Complexity Analysis:

If total() were not virtual, dynamic_cast could be used to identify the actual bill type and call its corresponding function.

For k different bill types, the program may need up to k type checks.

- Virtual function approach: O(1) dispatch per bill.
- dynamic_cast approach: Up to O(k) type checks per bill.

The virtual-function approach is more suitable when many different bill types are processed through one common interface.

TASK A3 – DESIGN WITH ABSTRACT CLASSES

A3.1 – Payer Interface:

An abstract class named Payer is designed to represent different payment methods.

It contains a pure virtual function:

virtual double amountCovered(const Bill&) const = 0;

The derived classes are:

1. SelfPay

- The patient pays the full bill amount.
- It returns the complete bill total.

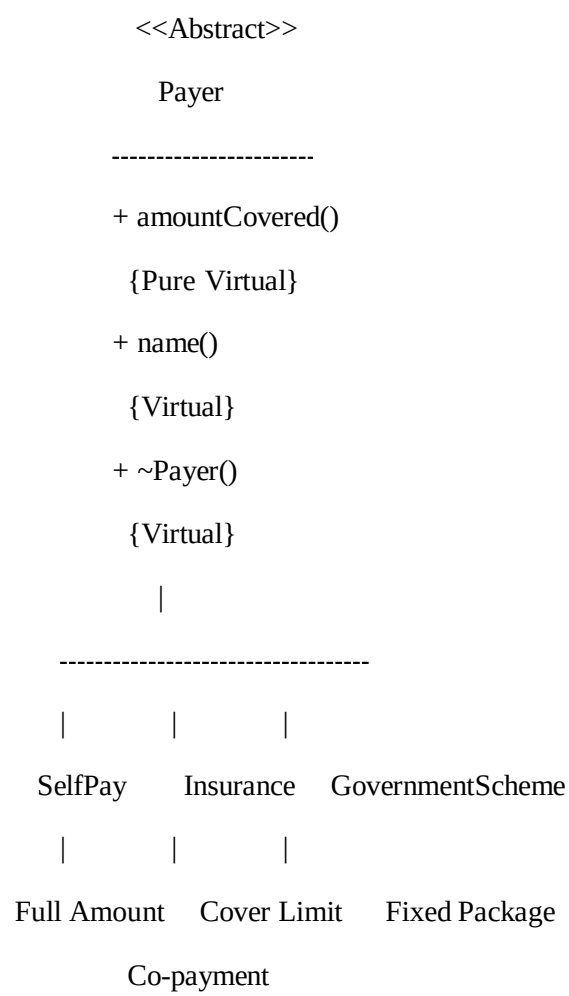
2. Insurance

- Stores the insurance cover limit.
- Stores the co-payment percentage.
- Calculates the amount covered after applying the policy rules.

3. GovernmentScheme

- Uses fixed package rates.
- Covers the amount according to the approved government package.

UML Diagram:



A3.2 – Extension:

A new payer called CorporatePayer can be added by inheriting from the existing Payer class.

It can implement:

- amountCovered()
- name()

The existing payer classes do not need to be modified.

This demonstrates extensibility through runtime polymorphism.

Alternative Concept and Complexity Analysis:

- Instead of an abstract Payer class, function pointers or std::function can be used

to store payment policies.

- Each callback can contain the required policy information, such as cover limit and co-payment percentage.
- Both approaches generally provide O(1) time per policy call.
- The abstract-class approach stores policy data in class members and supports inheritance. The function approach stores a callable object and may also store captured policy data.

TASK A4 – DESIGN THE EXCEPTION STRATEGY

A4.1 – Identify the Errors:

The hospital billing system may face the following errors:

1. Negative consultation or test charges.
2. Negative room or procedure charges.
3. Insurance claim exceeding the cover limit.
4. Unknown government package code.
5. Missing patient record.
6. Invalid text entered where a number is expected.
7. Memory allocation failure.
8. Invalid insurance policy values.

Exception Class Hierarchy:

```
          std::exception
               |
         std::runtime_error
               |
           BillingError
               |
    ----------------------------
    |          |          |
ClaimError  NegativeCharge  UnknownPackage
               |
```

MissingPatientError

Other standard exceptions:

- std::invalid_argument

- std::out_of_range

- std::bad_alloc

Specific exceptions provide clear information about the error.

For example:

- ClaimError indicates that a claim exceeds the allowed cover.
- NegativeChargeError indicates an invalid negative charge.
- UnknownPackageError indicates an unsupported package.
- MissingPatientError indicates that the patient record was not found.

A4.2 – Specify Exceptions:

The following functions may throw exceptions:

- Constructors that validate input.
- Bill creation functions.
- Claim approval functions.
- Patient record lookup functions.
- Government package lookup functions.
- Numeric conversion functions.
- Memory allocation operations.

Functions that perform simple operations and cannot fail should be designed not to throw.

The noexcept keyword can be used for functions that are guaranteed not to allow exceptions to escape.

Why should destructors not throw?

Destructors should not throw exceptions because they are called during object cleanup and stack unwinding.

If another exception is thrown while an exception is already being handled, the

program may terminate.

Therefore, destructors should perform cleanup safely and should not allow exceptions to escape.

Alternative Concept and Complexity Analysis:

Instead of exceptions, errors can be reported using return codes such as:

- SUCCESS
- INVALID_AMOUNT
- CLAIM_REJECTED
- PATIENT_NOT_FOUND

The function returns a status value, and the caller checks it.

Both approaches can take O(1) time to report an individual error.

Return codes require every function to check and pass error information to the next function. This increases the amount of code and creates a possibility that an error status may be ignored.

Exceptions separate error-handling logic from normal program execution.

TASK A5 – DESIGN RECOVERY AND RETHROWING

A5.1 – Where to Catch:

Exception handling should happen at suitable levels of the application.

Validation Layer:

- Checks the input.
- Throws the appropriate exception when invalid data is found.

Service Layer:

- Catches exceptions when additional information or logging is required.
- Uses throw; to rethrow the same exception.

Application Layer:

- Catches the final exception.
- Displays a suitable error message to the user.

A top-level catch(...) can be used to prevent unexpected exceptions from escaping the application.

A5.2 – Stack Unwinding:

Stack unwinding is the process in which local objects are automatically destroyed when an exception leaves a function or scope.

The destructors run in reverse order of object creation.

RAII stands for Resource Acquisition Is Initialization.

It connects resource management with object lifetime.

Examples include:

- std::string
- std::vector
- std::unique_ptr
- File stream objects

These objects automatically release their resources when they are destroyed, even when an exception occurs.

Therefore, RAII helps prevent memory leaks and resource-related errors.

Predicted Output:

~f ~main caught 42

Explanation:

1. The function f() creates a Guard object named g.
2. The function throws the integer value 42.
3. As the exception leaves f(), the destructor of g executes first.
4. The exception reaches the main function.
5. The Guard object named m is destroyed during stack unwinding.
6. The matching catch(int x) block catches the exception.
7. It displays caught 42.

The statement not printed does not appear because the exception transfers control directly to the catch block.

Final Output:

~f ~main caught 42

Alternative Concept and Complexity Analysis:

- If memory were released manually using delete in every catch block, every error-handling path would need separate cleanup code.
- The time required for cleanup remains proportional to the number of resources.
- However, manual memory management increases code size and maintenance effort.
- When a new error path is added, the programmer may forget to release a resource. This can cause memory leaks.
- RAII is safer because destructors automatically release resources.

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION

TASK B1 – IMPLEMENT AND VALIDATE THE POLYMORPHIC BILLING

B1.1 – Validate the Implementation:

The billing system is implemented using an abstract Bill class and derived classes.

The following bill types are tested:

1. OutPatientBill
2. InPatientBill
3. DayCareBill
4. EmergencyBill

The payer types are:

1. SelfPay
2. Insurance
3. GovernmentScheme
4. CorporatePayer

Manual Verification of Bill Totals:

Bill Type	Calculation	Total
OutPatient	500 + 1200	1700
InPatient	3 × 2500 + 40000	47500
DayCare	Package price	18000
Emergency	2000 + 500 × 4	4000

Daily Total:

1700 + 47500 + 18000 + 4000 = **85200**

The calculations are performed using the overridden total() functions.

Payer Verification Example

For a bill amount of 50,000:

Payer	Calculation	Amount Covered
SelfPay	Full bill amount	50000
Insurance	20% co-payment	40000
GovernmentScheme	Fixed package limit of 15000	15000
CorporatePayer	90% of bill	45000

The results show that different payer objects can process the same bill through a common interface.

B1.2 – Compare with Your Design:

The implementation follows the design created in Part A.

The abstract Bill class contains common functions, and each derived class implements its own bill calculation.

The Payer class provides a common interface for different payment methods.

Smart pointers such as unique_ptr are used to manage objects safely and reduce the possibility of memory leaks.

B1.3 – Prove Runtime Binding:

The virtual keyword is responsible for runtime binding.

When total() is called through a base-class pointer, the derived-class implementation is selected.

If total() is changed to a normal non-virtual function, runtime polymorphism will not work.

A pure virtual function must also be declared virtual. Removing the virtual keyword while keeping = 0 causes a compilation error.

B1.4 – Prove the Virtual Destructor:

The base class contains a virtual destructor.

It ensures that the derived-class destructor is called before the base-class destructor when

an object is deleted through a base-class pointer.

Removing the virtual destructor causes undefined behaviour.

The program should be tested using AddressSanitizer, and the actual output should be recorded.

Complete the Code – TODO 1 to TODO 3

Expected Output:

OP Ravi: 1700

IP Meena: 47500

Total 49200

Explanation:

- TODO 1 declares the pure virtual total() function and virtual type() function.
- TODO 2 calculates the OutPatient bill using consultation charges and test charges.
- TODO 3 calculates the InPatient bill using room charges and surgery charges.

The output confirms that runtime polymorphism selects the correct derived-class functions.

Removing virtual from total():

A pure virtual function must be virtual. Removing the keyword while keeping = 0 results in a compiler error.

Alternative Concept and Complexity Analysis:

Templates can be used to process each bill type separately without virtual function calls.

For example, separate containers can store OutPatient and InPatient objects.

The time complexity for processing n bills remains O(n).

The template approach avoids virtual dispatch, but it is difficult to store unrelated bill types in one common container. A mixed collection would require another solution such as std::variant.

TASK B2 – ENGINEER ERROR CONDITIONS

Error Testing:

The billing system should be tested using different invalid inputs.

Test Case	Input	Expected Behaviour
Negative charge	-500	Throw invalid_argument
Claim above cover	60000 with 50000 cover	Throw ClaimError
Unknown package	Invalid package code	Throw UnknownPackageError
Invalid number	Text instead of number	Throw invalid_argument
Memory allocation failure	Extremely large allocation	Handle bad_alloc if thrown

Explanation:

These tests verify that the program identifies invalid input and reports meaningful errors.

The program should also use a top-level exception handler to prevent unexpected errors from terminating normal application handling.

A very large allocation request does not guarantee that every operating system will immediately throw std::bad_alloc. Record the actual result of the controlled test.

Predicted Output:

ok done caught negative [log too large] caught too large

Explanation:

1. The first value is 100. It is valid, so ok is printed.
2. The program then prints done.
3. The second value is -5. It throws std::invalid_argument.
4. The process() function does not catch this exception type.
5. The main function catches it and prints caught negative.
6. The third value is 2e6, which is greater than 1e6.
7. The std::out_of_range handler inside process() catches and logs the error.
8. The throw; statement rethrows the same exception.
9. The main function catches it and prints caught too large.

Only the third exception is logged inside process() because the local handler catches only std::out_of_range.

Expected Output:

approved 40000

audit: exceeds cover

claim rejected: exceeds cover

error: negative amount

Explanation:

- TODO 1 checks whether the billed amount or cover amount is negative.
- If the billed amount exceeds the cover, it throws ClaimError.
- Otherwise, it returns the billed amount.
- TODO 2 prints the audit message and rethrows the same exception using throw;.

Difference Between throw; and throw e;

throw; rethrows the currently handled exception object and preserves its dynamic type.

throw e; creates a new exception object using the static type of the expression e.

If a derived exception is caught through a base-class reference, throw e; can cause object slicing.

Therefore, throw; is preferred when rethrowing the same exception.

Alternative Concept and Complexity Analysis:

std::optional<double> can be used instead of exceptions.

If a claim is approved, the function returns the approved amount. If the claim is rejected, it returns std::nullopt.

All approaches require O(n) time to process n claims.

The comparison should be performed for:

- 0% invalid claims.
- 1% invalid claims.
- 50% invalid claims.

Exceptions are generally efficient when no exception is thrown. However, repeatedly throwing and catching exceptions can be expensive.

Optional stores a value and an additional state indicating whether a value is available. Its exact memory size depends on the implementation.

TASK B3 – VIRTUAL FUNCTIONS VS SWITCH VS EXCEPTIONS: THE COST

B3.1 – Correctness on Small Data:

The same 10 bills are processed using two approaches:

1. Virtual functions.
2. A single structure with a type field and switch statement.

Both approaches should calculate the same bill totals.

The results are compared to verify correctness.

B3.2 – Increase the Data Size:

The execution time is measured for:

- 1,000 bills.
- 1,000,000 bills.
- 10,000,000 bills.

The programs should be compiled using the -O2 optimization option.

Compilation Command:

```
g++ -std=c++17 -O2 -Wall -Wextra Benchmark.cpp -o benchmark
```

Execution Command:

```
./benchmark
```

The benchmark should be executed multiple times. The median execution time should be recorded to reduce the effect of temporary system load.

B3.3 – Cost of Exceptions:

The time required to validate 1,000,000 bills should be measured using:

1. Return codes.
2. Exception handling.

The tests should use three invalid-input percentages.

Invalid Claims	Return Code	Exceptions
0%	Measure time	Measure time
1%	Measure time	Measure time
50%	Measure time	Measure time

The same input data should be used for both approaches.

When invalid claims are frequent, exception handling may take more time because throwing and catching exceptions involves additional work.

B3.4 – Engineering Trade-Off

Virtual Functions:

- Suitable when new bill types are added frequently.
- Supports a common interface.
- Reduces the need for central switch statements.

Switch Statement:

- Simple for a small number of fixed bill types.
- Easy to understand.
- Requires modification when new bill types are added.

std::variant and std::visit:

- Suitable when the available types are known in advance.
- Supports multiple alternatives in one container.
- Stores enough space for the largest alternative, along with a discriminator and possible padding.

Exception Handling:

- Suitable for unexpected errors.
- Keeps error-handling code separate from normal execution.
- Repeated exceptions can be expensive.

Complexity Analysis:

For n bills:

- Virtual approach: O(n).
- Switch approach: O(n).
- Variant approach: O(n).

For a fixed number of bill types, each approach has constant-time dispatch per bill in Big-O terms.

Actual execution performance must be determined through measurements.

Alternative Concept – std::variant

Instead of virtual functions or a switch statement, std::variant can store different bill types.

The std::visit() function can execute the correct calculation for the stored bill type.

This approach is useful when the number of possible bill types is known and fixed.

The memory required for a variant is approximately the size of its largest alternative plus a discriminator and alignment padding. The exact size depends on the compiler.

TASK B4 – FINAL ENGINEERING DECISION

B4.1 – Implement a Practical Extension:

A new bill type called MaternityBill is added to the existing system.

It calculates the bill using:

Total = Package Price + Additional Charges:

A new payer called CorporatePayer is also added.

It calculates the amount covered using a fixed corporate payment percentage.

Both classes inherit from the existing abstract classes.

No existing bill or payer class needs to be modified.

This demonstrates that the system supports extension without changing existing working classes.

B4.2 – Check Resource Safety:

AddressSanitizer is used to identify memory-related problems.

Compilation Command:

g++ -std=c++17 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer RAIIAndRethrow.cpp -o raii_test

Execution Command:

./raii_test

The program should be tested with both normal execution and exception handling.

The actual AddressSanitizer output should be included as evidence.

A clean sanitizer report means no memory error was detected during that particular

execution. It does not prove that every possible execution path is error-free.

B4.3 – Production Decision:

The polymorphic design with exception handling is suitable for the hospital billing system.

The main reasons are:

1. New bill types can be added without modifying existing bill classes.
2. New payment methods can be introduced through the Payer interface.
3. Runtime polymorphism supports different objects through a common interface.
4. Exception handling provides clear error reporting.
5. RAII and smart pointers help prevent memory leaks.
6. The system can be extended as the hospital network grows.

However, exceptions should be used mainly for unexpected failures such as invalid input, missing records and infrastructure problems.

If claim rejection is a frequent normal business outcome, a status or optional result may be more suitable.

A real hospital billing system must also include secure patient data handling, audit logs, transaction management, access control and extensive testing.

Alternative Concept – Configuration and Plug-ins:

Bill rates and payer limits can be stored in configuration files instead of creating a new C++ class for every rate change.

The system reads the rules and calculates the bill using those values.

Advantages:

- Rates can be updated without changing the main program.
- Configuration changes are easier to manage.
- Common billing rules can be reused.

Limitations:

- Rules must be validated.
- Invalid configuration may cause incorrect calculations.
- Complex new behaviour may still require a new plug-in or code change.

Configuration-based rules require additional lookup and interpretation work. Virtual functions provide direct class-based behaviour and are suitable for more complex extensions.

BENCHMARK RESULTS:

Important: Complete the following tables after executing the benchmark programs on your own computer. Enter actual measured values.

Bill Processing Time:

Number of Bills	**Virtual Functions**	**Switch Statement**	**std::variant**
1,000	_____	_____	_____
1,000,000	_____	_____	_____
10,000,000	_____	_____	_____

Exception Handling Time:

Invalid Claims	**Return Codes**	**Exceptions**
0%	_____	_____
1%	_____	_____
50%	_____	_____

Interpretation:

The benchmark results should be compared to identify which approach performs better for the tested data size.

The virtual approach supports easy extension. The switch approach may be suitable for a small fixed number of types. The variant approach is suitable for a known set of alternatives.

Exception handling is suitable for unusual errors, while return codes or optional values can be useful for frequent expected rejections.

COMPILATION AND EXECUTION COMMANDS

Check the compiler version:

g++ --version

Compile a normal C++ program:

g++ -std=c++17 -Wall -Wextra -pedantic filename.cpp -o program

Compile with AddressSanitizer:

g++ -std=c++17 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer filename.cpp -o program

Compile benchmark program:

g++ -std=c++17 -O2 -Wall -Wextra Benchmark.cpp -o benchmark

Run the program:

./program

On Windows PowerShell, run the executable using:

.\program.exe

GITHUB SUBMISSION CHECKLIST

The repository should contain:

1. Task01 – Bill Hierarchy.
2. Task02 – Runtime Binding.
3. Task03 – Payer Interface.
4. Task04 – Exception Strategy.
5. Task05 – Recovery and Rethrowing.
6. Task06 – Exception Handling Programs.
7. Task07 – Benchmark Programs.
8. README.md.
9. Input files and output screenshots.
10. Assignment report.

The README file should include:

- Student name.
- Register number.
- Compiler name and version.

- Compilation commands.
- Execution steps.
- Short explanation of each task.

At least one commit should be made for every task.

The working GitHub repository link must be included on the cover page and submitted to the faculty.

CONCLUSION:

This assignment demonstrates the importance of runtime polymorphism and exception handling in developing reliable C++ applications.

Abstract classes and virtual functions allow different bill and payer types to work through common interfaces. Exception handling helps identify and manage errors, while RAII ensures that resources are released safely.

The implementation and performance experiments help compare virtual functions, switch statements, variants and different error-handling methods.

The final design provides flexibility, reliability and easy extension for a growing hospital billing system.